ДГТУ · Введение в машинное обучение

Пара 1
Глубокий вводный разбор

Конспект для преподавателя к презентации Reveal.js

Главный вопрос пары: как из задачи и данных получить модель, которой можно доверять на новых объектах?

90 минут · 30 слайдов · русскоязычная онлайн-лекция

Версия: сентябрь 2026. Рейтинговая система курса находится в разработке.

Содержание

1 Педагогическая цель и маршрут **2**

2 Вступление: о курсе и пользе базы классического ML **2**

2.1 Слайд о работодателях: говорить точно 2

3 Что такое ML и когда оно вообще нужно **3**

4 С учителем и без: две разные постановки **3**

4.1 Обучение с учителем 4

4.2 Обучение без учителя 4

5 Карта алгоритмов без ложной гонки за «лучшим» **4**

6 Сквозной пример: цена квартиры **5**

6.1 Признак и утечка 5

6.2 Эмпирическая ошибка 6

7 Обобщение, проверка и выбор метрики **6**

7.1 Обучение и предсказание 6

7.2 Почему train недостаточно 6

7.3 Baseline и метрика 7

8 Заключение и первая практика **7**

9 Шпаргалка преподавателя: частые ошибки и точные ответы **8**

10 Быстрая памятка по каждому слайду **8**

11 Источники и границы использования **9**

1. Педагогическая цель и маршрут

После занятия студент должен без подсказки: (1) различить обучение с учителем и без него; (2) объяснить, чем регрессия отличается от классификации, а кластеризация — от обеих; (3) назвать объект, признаки и target; (4) объяснить разницу между обучением и предсказанием; (5) обосновать, зачем нужны отложенная проверка, baseline и метрика; (6) распознать очевидную утечку данных. Это не лекция о подробной реализации KNN, деревьев или бустинга. Алгоритмы сегодня нужны как карта различных способов строить зависимость.

Время	Слайды	Смысл и действие преподавателя
0–8 мин	1–3	Очень коротко о курсе и мотивации; не зачитывать программу недель.
8–25 мин	4–9	Что такое ML; когда он уместен; supervised/unsupervised; математическая цель; регрессия и классификация.
25–43 мин	10–15	Кластеризация; семейства методов и их предположения; вопрос аудитории.
43–61 мин	16–20	Цена квартиры: постановка, таблица, признаки, leakage, формальная выборка, эмпирический риск.
61–80 мин	21–27	fit/predict, обобщение, train/validation/test, ловушки split, baseline, метрика, параметры.
80–90 мин	28–30	Полный pipeline, следующая практика, контрольные вопросы.

Тайминг ориентировочный: если обсуждение разворачивается, сократите обзор списка алгоритмов, но не блок о train/test и утечках. В HTML у каждого слайда есть краткие speaker notes. Этот документ — более подробная опора для подготовки и живого объяснения.

2. Вступление: о курсе и пользе базы классического ML

0–8 мин

Что сказать на титульном слайде

«Сегодня я не буду начинать с формулы или с кода. Сначала очень коротко скажу, чему должен научить курс, а затем мы разберём язык машинного обучения: какие бывают задачи, что такое обучающие данные, что реально происходит при вызове `fit()` и почему хороший результат на знакомых примерах ничего не гарантирует».

Формат курса: 15 учебных недель, одна онлайн-лекция и одна онлайн-практика в неделю, экзамен в январе. Основной фокус — классическое ML на табличных данных и способность аргументированно ответить на базовые вопросы уровня ML Intern / Junior Data Scientist. Точные баллы и рейтинговая система ещё не утверждены; не называйте числа. Это достаточно проговорить устно за 1–2 минуты: пользователь специально убрал слайды с подробной программой и организацией.

2.1. Слайд о работодателях: говорить точно

У Яндекса в публичном гайде для ML-стажёров прямо названы классическое машинное обучение, стандартные алгоритмы и offline-метрики. SberDevices публикует отдельную техническую секцию «Классический ML» и перечисляет разбиение данных, метрики, оптимизацию, линейную регрессию, деревья, случайный лес и бустинг. Т-Банк в описании стажировки ML-инженера рекомендует знание линейной регрессии, бэггинга, бустинга и основ статистики;

на карьерной странице есть материал о ML-интервью.

VK, Сбер, Альфа-Банк, Ozon, Авито и МТС показаны как примеры крупных работодателей с направлениями работы с данными и ML. **Не утверждайте**, что у всех и на любой позиции гарантированно есть одинаковая секция classic ML. Отбор меняется по роли, команде и времени. Смысл слайда — показать карьерную применимость базы, а не дать «официальный список вопросов» всех компаний. Из публичных примеров направлений: VK Education Practice по AI и анализу данных, стажировки Сбера по DS/AI, Data Science в стажировках Альфа-Банка.

Переход

«Почему на интервью спрашивают не только код? Потому что библиотека может вернуть число даже тогда, когда задача поставлена неверно, данные протекли из будущего или метрика не отражает реальную пользу. Теперь разберём этот путь с самого начала».

3. Что такое ML и когда оно вообще нужно

8–15 мин

Обычная программа содержит правила, определённые разработчиком. **Обучение по данным** выбирает конкретную зависимость из семейства допустимых моделей, используя наблюдаемые примеры. При этом ML не заменяет постановку задачи: человек по-прежнему выбирает, что считать объектом, какие данные использовать, какую ошибку считать критичной и какое действие предпринять после прогноза.

Сравнение, которое можно произнести: вычислить возраст по дате рождения — простое правило; предсказать вероятность ухода клиента по истории поведения — кандидат для ML. Не противопоставляйте ML и правила абсолютно: модель сама является программой, а признаки и пороги часто включают ручные решения.

Вопрос аудитории

«Если формула стоимости доставки заранее известна и стабильно работает, зачем нам обучать модель?» Ожидаемый ответ: незачем, если правило покрывает задачу и удовлетворяет требованиям. ML добавляет сложность, стоимость поддержки и риск ошибок.

Перед выбором алгоритма задайте три вопроса. **Есть ли полезное решение после прогноза? Есть ли репрезентативные, допустимые к использованию данные? Есть ли способ сравнить результат с простым решением?** Если один ответ отрицательный, «попробуем бустинг» не является полноценным планом.

4. С учителем и без: две разные постановки

15–30 мин

4.1. Обучение с учителем

Для каждого учебного объекта есть пара (x_i, y_i): описание x_i и известный ответ y_i. Цель — построить правило f, которое на новом x выдаёт прогноз $\hat{y} = f(x)$. «Учитель» здесь означает метки в данных, а не человека, который стоит над алгоритмом.

Идеальная математическая цель:

$$f^* = \arg\min_{f \in \mathcal{F}} \mathbb{E}_{(x,y)\sim P}\big[L(f(x), y)\big].$$

Здесь $\mathcal{F}$ — семейство кандидатов; L — цена ошибки на одном объекте; P — неизвестное распределение будущих случаев. Объясните словами: мы хотим, чтобы модель ошибалась мало *на будущих объектах*, а не только на тех, которые уже лежат в таблице. Распределение P неизвестно, поэтому используем конечную выборку как его приближение. Если будущее существенно изменится, гарантия переноса становится слабее.

Регрессия: ответ — численная величина, например цена квартиры или время доставки. Обычно пишут $\hat{y} \in \mathbb{R}$. **Классификация**: ответ принадлежит конечному набору классов, например спам/не спам, $\hat{y} \in \{0, \dots, K-1\}$. Классификатор может выдавать также вероятности классов; окончательный класс получается после правила принятия решения, часто с порогом. Прогноз рейтинга 1–5 может допускать несколько постановок — важны смысл шкалы и метрика.

Не путать задачу и алгоритм

«Классификация» и «регрессия» отвечают на вопрос, *какой результат нужен*. «KNN», «линейная модель» и «дерево» отвечают на вопрос, *как строить предсказание*. Дерево может быть классификатором или регрессором; KNN — тоже.

4.2. Обучение без учителя

У объектов есть признаки x_i, но нет заданного целевого ответа y_i. Алгоритм ищет структуру: **кластеризация** группирует похожие объекты, **снижение размерности** даёт более компактное представление, **обнаружение аномалий** ищет необычные случаи. K-Means и DBSCAN — два алгоритма кластеризации; PCA — метод снижения размерности. Не представляйте PCA как разновидность классификации.

Кластер не тождествен классу: он возникает из выбора признаков, масштаба и понятия близости. Разбив клиентов по сумме покупок и частоте посещений, можно получить одни группы; по категориям товаров — другие. Название «лояльные клиенты» даёт аналитик после исследования, а не алгоритм как объективную истину.

Проверка понимания перед следующей частью

«Длительность поездки по размеченной истории — что это? Спам по размеченным письмам? Группы клиентов без меток?» Ожидаемые ответы: регрессия; классификация; кластеризация. Попросите объяснить признак выбора, а не просто назвать термин.

5. Карта алгоритмов без ложной гонки за «лучшим»

30–43 мин

У каждого семейства есть своё полезное предположение о данных. **KNN**: близкие в выбранной метрике объекты часто имеют похожие ответы; результат зависит от масштабирования и выбора k. **Линейные модели**: прогноз выражается через взвешенную сумму признаков; сложные зависимости требуют подходящего представления признаков. **Деревья**: последовательные пороговые вопросы разбивают пространство; одиночное глубокое дерево может быть нестабильным и переобучаться. **Случайный лес**: усредняет множество различающихся деревьев, снижая их нестабильность. **Бустинг**: строит ансамбль последовательно, добавляя модели, которые исправляют ошибки текущего решения. **SVM**: ищет разделяющую границу с запасом; детали будут позже.

Нет универсально лучшего алгоритма. Уместность зависит от объёма и природы данных, требований к интерпретации, скорости предсказания, стоимости ошибок и результатов честной проверки. На первой лекции достаточно показать принципы, а не выводить каждый метод.

Как сказать про `fit()`

«Интерфейс библиотеки выглядит одинаково: `fit(X,y)`. Но за ним скрыты разные действия. KNN в основном сохраняет примеры; линейная модель оценивает коэффициенты; дерево выбирает пороги разбиений. Команда одна, математическая процедура разная».

6. Сквозной пример: цена квартиры

43–61 мин

Теперь пример появляется после карты задач. Сначала спросите: *«Цена квартиры — это какой тип задачи?»* Ответ: регрессия с учителем, если есть история сделок с известными ценами. Затем покажите объект: квартира площадью 62 м^2, 7-й этаж, 12 минут до метро. Попросите назвать недостающие сведения: район, состояние, год постройки, дата сделки, тип дома. Три признака намеренно недостаточны.

В таблице строка — **объект**, столбцы характеристик — **признаки**, известная цена — **целевая переменная** или target. Запись

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$$

означает n пар: x_i — вектор характеристик i-й квартиры, y_i — её цена. Индекс i только нумерует примеры. Для новой квартиры есть x_{new}, но её y_{new} пока неизвестен. В презентации три сделки показаны только для объяснения — этого объёма недостаточно для настоящего обучения.

6.1. Признак и утечка

Главный тест для любого признака: *«Будет ли он доступен в момент реального прогноза?»* Площадь и этаж обычно известны; финальная цена сделки — ответ, а не признак. Информация, появившаяся после продажи, тоже недопустима для прогноза до продажи. Если она попадёт в X, качество в ноутбуке может быть фантастическим, а применение модели — невозможным. Это **утечка данных**. Отдельная опасность: один и тот же объект или почти дубликаты оказываются и в обучении, и в проверке.

6.2. Эмпирическая ошибка

Поскольку неизвестное распределение будущего P нам недоступно, многие методы ориентируются на ошибку в доступных данных:

$$\hat{R}(f) = \frac{1}{n}\sum_{i=1}^{n} L\big(f(x_i), y_i\big).$$

Это средняя потеря на n обучающих примерах. Формула описывает общую идею оптимизации, но **не утверждает**, что каждый алгоритм буквально обучается градиентным спуском: KNN устроен иначе. Минимум $\hat{R}$ — не конец истории. Можно добиться почти нулевой ошибки, запомнив шум, и плохо работать на новых квартирах.

7. Обобщение, проверка и выбор метрики

61–80 мин

7.1. Обучение и предсказание

При обучении алгоритм видит признаки и ответы: `model.fit(X_train, y_train)`. При применении он получает признаки нового объекта: `model.predict(X_new)`. Общее обозначение прогноза:

$$\hat{y} = f(x; \theta).$$

Шляпка над y означает прогноз, а не истинный ответ. θ обозначает обученное состояние модели в широком смысле; это не обязательно числовой вектор весов для каждого алгоритма. У линейной модели частный случай: $\hat{y} = w^{\mathsf{T}}x + b$, где w и b находятся из данных. Число соседей k в KNN — гиперпараметр: его задают до конкретного обучения и выбирают по корректной проверке.

7.2. Почему train недостаточно

Цель — обобщающая способность: работа на новых объектах. Проверка на тех же строках, которые модель видела при fit, измеряет главным образом способность объяснить знакомые данные. Низкая ошибка на train и высокая на test может говорить о переобучении, утечке при проверке или сдвиге данных; она не доказывает единственную причину. Высокая ошибка и на train, и на test может указывать на слишком слабую модель, плохие признаки или неправильную постановку.

Разделение данных служит разным ролям: **train** — обучение и построение baseline; **validation** — выбор модели, признаков и гиперпараметров; **test** — заключительная оценка уже выбранного решения. Если многократно смотреть на test и менять модель, он фактически превращается в ещё одну validation-выборку.

Честный split должен имитировать реальное применение. Для будущих сделок часто важен временной порядок: обучение на прошлом, проверка на более позднем. Для повторяющихся клиентов, домов или устройств требуется учитывать группы, чтобы одни и те же сущности не подсказали ответ с обеих сторон split. Случайное разбиение не универсально.

7.3. Baseline и метрика

Константный baseline для цены квартиры — средняя цена *обучающих* сделок:

$$\hat{y}_{\text{base}} = \frac{1}{n}\sum_{i=1}^{n} y_i.$$

Его применяют к отложенным объектам и сравнивают с более сложной моделью на одной и той же метрике. Для MAE на m проверочных объектах:

$$\text{MAE} = \frac{1}{m}\sum_{j=1}^{m} |y_j - \hat{y}_j|.$$

MAE выражается в тех же единицах, что и цена. Например, ошибки 1, 2 и 3 млн рублей дают MAE 2 млн рублей. Этот числовой пример полезен, потому что студенты сразу видят смысл усреднения. Baseline на среднем не обязательно лучший: при некоторых метриках более подходяща медиана. Если сложная модель хуже baseline, задайте вопрос не «какой ещё алгоритм добавить?», а «не ошиблись ли мы с данными, признаками, разделением или метрикой?».

Функция потерь даёт сигнал для настройки параметров; **метрика** сообщает, насколько результат полезен по выбранному критерию. Это различие ролей, а не требование двух разных формул. Для задачи мошенничества простая accuracy может быть обманчива: при редких положительных случаях модель «всегда нет» имеет высокую долю правильных ответов, но не решает задачу. Здесь пока не выводите precision и recall — лишь укажите, что цена ложной тревоги и пропущенного случая разная.

Мини-разбор у доски

Пусть истинные цены трёх квартир: 8, 10, 12 млн, а прогнозы: 9, 8, 13 млн. Абсолютные ошибки: 1, 2, 1 млн. MAE $= (1+2+1)/3 \approx 1{,}33$ млн. Попросите студентов самостоятельно назвать единицу измерения и подумать, как оценить, достаточно ли это хорошо: нужен контекст рынка и baseline.

8. Заключение и первая практика

80–90 мин

Пройдите полную цепочку ещё раз, но теперь студенты должны объяснять каждый шаг: **задача** (цена), **данные** (история сделок), **разделение** (имитация будущего), **baseline** (средняя цена), **модель** (например, линейная), **метрика** (MAE), **анализ ошибок** (где промахиваемся), **вывод** (что изменить и почему). Цепочка итеративна: обнаружив проблемы, мы можем вернуться к задаче и данным.

На первой практике студенты в Jupyter загрузят небольшой датасет, выделят X и y, разделят данные, обучат baseline, получат предсказания и посчитают метрику. До неё им следует проверить Python, Jupyter, Git и доступ к GitHub. При проблемах с локальной средой Colab можно временно использовать как резерв.

Три финальных вопроса аудитории

1. Цена квартиры — задача с учителем или без, регрессия или классификация?
2. Группы клиентов без меток — чем отличаются от заранее заданных классов?
3. Почему высокое качество на train не доказывает, что модель хорошо работает?

9. Шпаргалка преподавателя: частые ошибки и точные ответы

Неверное утверждение / вопрос	Что ответить
«ML сам найдёт правильную задачу»	Нет: target, доступные признаки, критерий качества и действие после прогноза задаются отдельно.
«Классификация — это алгоритм»	Это тип задачи. Её можно решать разными моделями.
«Кластер — это настоящий класс»	Кластер зависит от выбранных признаков, масштаба, метрики и метода; смысл требует отдельной интерпретации.
«`fit()` всегда меняет веса»	Нет: у KNN он в основном сохраняет примеры; у линейной модели оценивает коэффициенты; дерево строит разбиения.
«0 ошибок на train — идеальная модель»	Это может быть переобучение или утечка. Проверяем на данных, не участвовавших в обучении и подборе.
«Можно подобрать модель по test»	Тогда test перестаёт быть независимой заключительной проверкой. Выбор делаем по validation или CV.
«Baseline должен быть сложным»	Напротив: это понятная простая точка сравнения, которую трудно ошибочно улучшить только на бумаге.
«Метрика всегда равна loss»	Иногда формулы совпадают, иногда нет; различайте роль при оптимизации и роль при оценке полезности.
«Какой алгоритм лучший?»	Без постановки, данных, метрики и ограничений такой вопрос не имеет содержательного ответа.

10. Быстрая памятка по каждому слайду

Эта таблица дополняет развёрнутые объяснения выше. Она предназначена для последней репетиции и быстрого взгляда во время пары; не зачитывайте её студентам как текст.

№	Что подсветить	Что спросить или как перейти
1	Главный вопрос: модель на новых данных.	Пообещайте сначала контекст, потом формулы.
2	Цель курса: осмысленное решение, не вызов API.	«Что делает ответу модели доверие?»
3	Яндекс и SberDevices прямо описывают ML-секции; у остальных формат зависит от команды.	Не обещайте одинаковый отбор во всех компаниях.
4	ML подбирает зависимость по примерам; человек задаёт задачу.	«Что всё равно остаётся за человеком?»
5	ML не нужен, если достаточно надёжного правила.	«Зачем обучать модель для вычисления возраста?»
6	У supervised есть известные y, у unsupervised — нет.	«Кто здесь «учитель»?»
7	Цель — ошибка на будущем распределении, а не только на таблице.	Прочитайте смысл f, L, P, не выводя формулу.
8	Регрессия даёт число; классификация — класс.	Попросите пример каждого типа.
9	Тип задачи не равен названию алгоритма.	«Может ли дерево предсказывать цену?»
10	В unsupervised есть разные цели: группы, проекции, аномалии.	«Что здесь может быть target?»
11	Кластер зависит от признаков и близости; это не размеченный класс.	«Поменяем признаки — изменятся ли группы?»

№	Что подсветить	Что спросить или как перейти
12	У KNN, линейной модели, дерева, ансамбля, SVM разные механизмы.	Не объявляйте «победителя».
13	Индуктивные предположения определяют поведение модели.	«Что предполагает KNN о соседях?»
14	K-Means/DBSCAN группируют; PCA снижает размерность.	Не называйте PCA классификатором.
15	Три задачи для диагностики; ответы появляются после клика.	Попросите обоснование каждого ответа.
16	Цена квартиры — регрессия с учителем.	«Каких признаков не хватает?»
17	Строка — объект, столбцы — признаки, цена — target.	«Что неизвестно для новой квартиры?»
18	Признак допустим, если доступен в момент прогноза.	«Можно ли взять финальную цену сделки в X?»
19	Набор из n пар (x_i, y_i).	Найдите x и y в таблице.
20	Средняя ошибка на train — приближение желаемого качества на будущем.	«Почему минимум на train может обмануть?»
21	`fit` видит ответы; `predict` их не получает.	«Что означает шляпка над y?»
22	Запоминание и обобщение — разные достижения.	«Что скажет высокий train и низкий test?»
23	Train, validation, test имеют разные роли.	«Где выбрать число соседей k?»
24	Разбиение должно имитировать время и группы реальной задачи.	«Можно ли обучить на будущем и проверить на прошлом?»
25	Baseline — среднее train; MAE — средняя абсолютная ошибка test.	Посчитайте $(1 + 2 + 1)/3$ млн.
26	Метрика зависит от цены ошибок и действия после прогноза.	«Почему accuracy может обмануть при редких случаях?»
27	Веса линейной модели учатся; k KNN задаётся заранее.	«Что параметр, а что гиперпараметр?»
28	Полный ML pipeline итеративен.	Пройдите каждый шаг на квартире.
29	На практике соберём pipeline в Jupyter.	Напомните проверить среду.
30	Три вопроса на выходе: тип задачи, кластеры, train/test.	Завершите без новой терминологии.

11. Источники и границы использования

1. Яндекс, «Тренировки по ML 1.0», первая лекция. Методический ориентир: формальная постановка supervised learning и объяснение через конкретные алгоритмы; здесь материал переработан под отдельную вводную пару.
2. Описание первой лекции на Яндекс Образовании.
3. Яндекс: как нанимают ML-стажёров. Основание для упоминания classic ML в отборе.
4. SberDevices: интервью ML-разработчика. Основание для отдельной секции «Классический ML».
5. Т-Банк: стажировка ML-инженера. Основание для упоминания линейной регрессии, бэггинга, бустинга и статистики как полезной базы.
6. VK: практика по AI и анализу данных; Сбер: направления стажировок; Альфа-Банк: направления стажировок. Эти ссылки подтверждают наличие направлений, но не единый формат интервью.